IS455 Midterm Project - Chicago Crime and 311 Analytics

Course: IS455 - Database Design & Prototyping
Submitted: March 2026

This project analyzes Chicago crime incidents and Chicago 311 service requests using SQL analytics and a DuckDB data pipeline.

The goal is to apply database design and data analysis concepts from the IS455 course.

Project Goal

- This project uses Chicago crime incidents and a Chicago 311 service-request extract to answer four questions:

1. **What are the most common crime types and their arrest rates?**
2. **Does the city's 311 service request backlog correlate with crime volume on the same day?**
3. **Which districts and location types have the highest crime concentration?**
4. **How do daily crime counts vary by day of week or month?**

Syllabus Concepts Used

- DuckDB reading CSVs directly
- `GROUP BY`, `COUNT`, `AVG`, `SUM`
- duplicate detection and null-rate audits
- join-cardinality debugging and pre-aggregate-then-join
- practical normalization with dimensions and fact tables
- idempotent ETL with `CREATE OR REPLACE TABLE AS`
- Parquet export and CSV vs Parquet benchmarking
- query-plan inspection with `EXPLAIN`
- run manifest for reproducibility

Section 0 - Setup

In [18]:
```
import duckdb
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import os
import warnings

warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid', palette='muted')

DB_PATH = 'chicago_crime.duckdb'
con = duckdb.connect(DB_PATH)
print(f'Connected to {DB_PATH}')

con.execute("""
    CREATE OR REPLACE TABLE dim_police_district_name AS
    SELECT * FROM (
        VALUES
            (1,  'Central'),
            (2,  'Wentworth'),
            (3,  'Grand Crossing'),
            (4,  'South Chicago'),
            (5,  'Calumet'),
            (6,  'Gresham'),
            (7,  'Englewood'),
            (8,  'Chicago Lawn'),
            (9,  'Deering'),
            (10, 'Ogden'),
            (11, 'Harrison'),
            (12, 'Near West'),
            (14, 'Shakespeare'),
            (15, 'Austin'),
            (16, 'Jefferson Park'),
            (17, 'Albany Park'),
            (18, 'Near North'),
            (19, 'Town Hall'),
            (20, 'Lincoln'),
            (22, 'Morgan Park'),
            (24, 'Rogers Park'),
            (25, 'Grand Central')
    ) AS t(district, district_name)
""")
print('Created dim_police_district_name lookup table')
```

```
Connected to chicago_crime.duckdb
Created dim_police_district_name lookup table
```

Section 1 - Data Download and Pipeline Build

I imported DuckDB, pandas, and visualization libraries.

Then I connected to a DuckDB database called chicago_crime.duckdb.

I also create a lookup table for police district names to make district analysis easier.

The data pipeline follows three layers:

Raw layer

- Loads CSV files for crimes and 311 requests.

Staging layer

- Cleans and standardizes data.

Curated layer

- Creates analytical tables using dimension and fact tables.

Fact tables include:

- crime events
- 311 requests
- daily 311 backlog. *The pipeline also checks join cardinality to avoid duplicate rows. *This notebook uses two CSV files in `data/raw`:

- `crimes.csv`
- `311_service_requests.csv`

The 311 extract below uses a Chicago 311 service-request endpoint and saves it under the generic project filename `311_service_requests.csv`.

In [19]:
```
import urllib.request

os.makedirs('data/raw', exist_ok=True)

DOWNLOADS = {
    'data/raw/crimes.csv': 'https://data.cityofchicago.org/resource/ijzp-q8t
    'data/raw/311_service_requests.csv': 'https://data.cityofchicago.org/res
}

for path, url in DOWNLOADS.items():
    if not os.path.exists(path):
        print(f'Downloading {path}...')
        urllib.request.urlretrieve(url, path)
        print(f'  Saved to {path}')
```

```
    else:
        print(f'  Already exists: {path}')
```

```
Already exists: data/raw/crimes.csv
Already exists: data/raw/311_service_requests.csv
```

In [15]:
```
%run build.py
```

```
[build.py] Connected to chicago_crime.duckdb

[STEP 1] Loading RAW layer...

  [AUDIT] Crimes (raw) -> raw_crimes
    rows         : 100,000
    null id          : 0
    dupe id          : 0

  [AUDIT] 311 requests (raw) -> raw_311
    rows         : 50,000
    null sr_number   : 0
    dupe sr_number   : 0

[STEP 2] Building STAGING layer...

  [AUDIT] Crimes (staging) -> stg_crimes
    rows         : 100,000
    null crime_id    : 0
    dupe crime_id    : 0

  [AUDIT] 311 requests (staging) -> stg_311
    rows         : 50,000
    null request_id  : 0
    dupe request_id  : 0

[STEP 3] Building CURATED dimension tables...

  [AUDIT] dim_crime_type -> dim_crime_type
    rows         : 30
    null crime_type_id: 0
    dupe crime_type_id: 0

  [AUDIT] dim_district -> dim_district
    rows         : 23
    null district_id : 0
    dupe district_id : 0

  [AUDIT] dim_location_type -> dim_location_type
    rows         : 119
    null location_type_id: 0
    dupe location_type_id: 0

[STEP 4] Building CURATED fact tables...

  [CARDINALITY CHECK] pre-join: 100,000 | post-join: 100,000
  OK: joins preserved the crime-event grain.

  [AUDIT] fact_crime_events -> fact_crime_events
    rows         : 100,000
    null crime_id    : 0
    dupe crime_id    : 0

  [AUDIT] fact_311_requests -> fact_311_requests
    rows         : 50,000
    null request_id  : 0
```

```
    dupe request_id  : 0

  [AUDIT] fact_311_daily -> fact_311_daily
    rows         : 2,200
    null activity_date: 0
    dupe activity_date: 0

[STEP 5] Exporting curated tables to Parquet...
  Exported -> data/curated/fact_crime_events.parquet
  Exported -> data/curated/fact_311_requests.parquet
  Exported -> data/curated/fact_311_daily.parquet
  Exported -> data/curated/dim_crime_type.parquet
  Exported -> data/curated/dim_district.parquet
  Exported -> data/curated/dim_location_type.parquet

[STEP 6] Writing run manifest...
 run_id                     run_ts  raw_crimes_rows  stg_crimes_rows  raw_31
1_rows  stg_311_rows  fact_crime_rows  fact_311_req_rows  fact_311_daily_row
s                                          notes
      9 2026-03-10 22:05:59.250793           100000           100000
50000         50000           100000              50000                 2200
Automated build via build.py (crimes + 311)
      8 2026-03-10 22:05:22.586273           100000           100000
50000         50000           100000              50000                 2200
Automated build via build.py (crimes + 311)
      7 2026-03-10 22:04:21.589931           100000           100000
50000         50000           100000              50000                 2200
Automated build via build.py (crimes + 311)

[build.py] Pipeline complete.
```

Section 2 - Raw Layer Audit

Before analysis, I perform data audits to check:

- row counts
- duplicate IDs
- null values.

Results show:

- 100,000 crime records
- 50,000 311 requests
- no duplicate keys or major missing values. *This ensures the data is reliable before running analysis. *These checks validate row counts, keys, null rates, and date ranges.

In [20]:
```
con.execute("""
    SELECT 'raw_crimes' AS table_name, COUNT(*) AS row_count FROM raw_crimes
    UNION ALL
```

```
    SELECT 'raw_311', COUNT(*) FROM raw_311
""").df()
```

Out[20]:

	table_name	row_count
0	raw_crimes	100000
1	raw_311	50000

In [21]:

```
# Crime raw-data audit: key nulls and duplicate keys
con.execute("""
    SELECT
        COUNT(*) AS total_rows,
        SUM(CASE WHEN id IS NULL THEN 1 ELSE 0 END) AS null_id,
        COUNT(*) - COUNT(DISTINCT id) AS duplicate_ids,
        SUM(CASE WHEN primary_type IS NULL THEN 1 ELSE 0 END) AS null_primar
        SUM(CASE WHEN district IS NULL THEN 1 ELSE 0 END) AS null_district,
        SUM(CASE WHEN location_description IS NULL THEN 1 ELSE 0 END) AS nul
    FROM raw_crimes
""").df()
```

Out[21]:

	total_rows	null_id	duplicate_ids	null_primary_type	null_district	null_location_de
0	100000	0.0	0	0.0	0.0	

In [22]:

```
# 311 raw-data audit: request key quality and missing values
con.execute("""
    SELECT
        COUNT(*) AS total_rows,
        SUM(CASE WHEN sr_number IS NULL THEN 1 ELSE 0 END) AS null_request_i
        COUNT(*) - COUNT(DISTINCT sr_number) AS duplicate_request_ids,
        SUM(CASE WHEN created_date IS NULL THEN 1 ELSE 0 END) AS null_creati
        SUM(CASE WHEN status IS NULL THEN 1 ELSE 0 END) AS null_status,
        SUM(CASE WHEN sr_type IS NULL THEN 1 ELSE 0 END) AS null_service_typ
    FROM raw_311
""").df()
```

Out[22]:

	total_rows	null_request_id	duplicate_request_ids	null_creation_date	null_status
0	50000	0.0	0	0.0	0.0

In [23]:

```
# Date-range audit across both datasets
con.execute("""
    SELECT 'crimes' AS dataset, MIN(date) AS earliest, MAX(date) AS latest
    FROM raw_crimes
    UNION ALL
    SELECT '311', MIN(created_date), MAX(created_date)
    FROM raw_311
""").df()
```

Out[23]:

	dataset	earliest	latest
0	crimes	2023-08-16T23:30:00.000	2023-12-31T23:59:00.000
1	311	2018-07-02T06:24:36.000	2026-02-22T22:54:12.000

Section 3 - Design Decisions

One key design decision is using two fact tables.

- Crime data is at the incident level.
- 311 data is analyzed at the daily level.
- To avoid many-to-many join errors, I use the pre-aggregate then join strategy.
- Both datasets are aggregated to one row per day before joining.

Why pre-aggregate before joining?

- The backlog question is a same-day question, so both sides are aggregated to one row per day before the join. That follows Week 3's pre-aggregate-then-join strategy.

In [24]:

```
# Cardinality-safe daily join demo
crime_days = con.execute('SELECT COUNT(DISTINCT event_date) FROM fact_crime_
joined_days = con.execute("""
    SELECT COUNT(*)
    FROM (
        SELECT event_date, COUNT(*) AS crime_count
        FROM fact_crime_events
        GROUP BY event_date
    ) c
    LEFT JOIN fact_311_daily d
        ON c.event_date = d.activity_date
""").fetchone()[0]

print(f'Distinct crime days before join: {crime_days}')
print(f'Rows after daily join:         {joined_days}')
print('Result: the daily grain is preserved.')
```

```
Distinct crime days before join: 138
Rows after daily join:         138
Result: the daily grain is preserved.
```

Section 4 - SQL Concepts from Lectures

Here I analyze crime types using SQL aggregation functions.

Functions used include:

- COUNT()

- SUM()
- ROUND()
- window function SUM() OVER().

These help calculate:

- total crimes
- percentage of total
- arrest counts
- arrest rate.

Key result:

- Theft is the most common crime.
- Weapons violations have the highest arrest rate, about 56%.

In [25]:
```
# Q1: GROUP BY + aggregation
crime_type_rates = con.execute("""
    SELECT
        crime_type,
        COUNT(*) AS total_crimes,
        ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS pct_of_total,
        SUM(CASE WHEN arrest_made THEN 1 ELSE 0 END) AS arrests,
        ROUND(SUM(CASE WHEN arrest_made THEN 1 ELSE 0 END) * 100.0 / COUNT(*
    FROM fact_crime_events
    WHERE crime_type IS NOT NULL
    GROUP BY crime_type
    ORDER BY total_crimes DESC
    LIMIT 10
""").df()

crime_type_rates
```

Out[25]:

	crime_type	total_crimes	pct_of_total	arrests	arrest_rate_pct
0	THEFT	22553	22.55	1176.0	5.2
1	BATTERY	16842	16.84	2670.0	15.9
2	CRIMINAL DAMAGE	11303	11.30	327.0	2.9
3	MOTOR VEHICLE THEFT	10592	10.59	316.0	3.0
4	ASSAULT	8705	8.71	892.0	10.2
5	DECEPTIVE PRACTICE	6083	6.08	254.0	4.2
6	OTHER OFFENSE	5912	5.91	977.0	16.5
7	ROBBERY	4957	4.96	271.0	5.5
8	WEAPONS VIOLATION	3001	3.00	1677.0	55.9
9	BURGLARY	2907	2.91	165.0	5.7

In [26]:

```
# Q2 and Q4 prep: daily crime counts with time buckets
daily_crime = con.execute("""
    SELECT
        event_date,
        COUNT(*) AS crime_count,
        STRFTIME(event_date, '%w') AS day_of_week_num,
        STRFTIME(event_date, '%A') AS day_of_week,
        STRFTIME(event_date, '%m') AS month_num,
        STRFTIME(event_date, '%B') AS month_name
    FROM fact_crime_events
    GROUP BY event_date
    ORDER BY event_date
""").df()

daily_crime.head()
```

Out[26]:

	event_date	crime_count	day_of_week_num	day_of_week	month_num	month_nar
0	2023-08-16	11	3	Wednesday	08	Augu
1	2023-08-17	783	4	Thursday	08	Augu
2	2023-08-18	720	5	Friday	08	Augu
3	2023-08-19	809	6	Saturday	08	Augu
4	2023-08-20	791	0	Sunday	08	Augu

In [27]:

```
# Q2: pre-aggregate then join daily crime totals to 311 backlog
crime_311_daily = con.execute("""
    WITH crime_daily AS (
```

```
        SELECT event_date, COUNT(*) AS crime_count
        FROM fact_crime_events
        GROUP BY event_date
    )
    SELECT
        c.event_date,
        c.crime_count,
        d.opened_requests,
        d.closed_requests,
        d.backlog_end_of_day
    FROM crime_daily c
    LEFT JOIN fact_311_daily d
        ON c.event_date = d.activity_date
    ORDER BY c.event_date
""").df()

corr_df = con.execute("""
    WITH crime_daily AS (
        SELECT event_date, COUNT(*) AS crime_count
        FROM fact_crime_events
        GROUP BY event_date
    )
    SELECT CORR(c.crime_count, d.backlog_end_of_day) AS crime_backlog_corr
    FROM crime_daily c
    INNER JOIN fact_311_daily d
        ON c.event_date = d.activity_date
""").df()

corr_df
```

Out[27]:

	crime_backlog_corr
0	-0.19532

Next I created daily crime counts and extract time features like:

- day of week
- month.

Then I analyzed the relationship between crime counts and 311 backlog using the SQL function CORR().

The correlation value is -0.19, which indicates very weak relationship between crime volume and service request backlog.

In [28]:

```
# Q3: district concentration
district_concentration = con.execute("""
    SELECT
        c.district,
        COALESCE(d.district_name, CONCAT('District ', c.district::VARCHAR))
        COUNT(*) AS total_crimes,
        ROUND(SUM(CASE WHEN c.arrest_made THEN 1 ELSE 0 END) * 100.0 / COUNT
    FROM fact_crime_events c
    LEFT JOIN dim_police_district_name d
```

```
        ON c.district = d.district
    WHERE c.district IS NOT NULL
    GROUP BY c.district, district_name
    ORDER BY total_crimes DESC
    LIMIT 10
""").df()

district_concentration
```

Out[28]:

	district	district_name	total_crimes	arrest_rate_pct
0	8	Chicago Lawn	6646	9.5
1	12	Near West	6165	7.1
2	6	Gresham	5868	10.8
3	4	South Chicago	5617	8.7
4	25	Grand Central	5486	11.9
5	1	Central	5476	17.2
6	11	Harrison	5442	24.1
7	19	Town Hall	5379	7.8
8	18	Near North	5105	13.9
9	2	Wentworth	4940	7.3

In [29]:

```
# Q3: location-type concentration
location_concentration = con.execute("""
    SELECT
        location_desc,
        COUNT(*) AS total_crimes,
        ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS pct_of_total
    FROM fact_crime_events
    WHERE location_desc IS NOT NULL
    GROUP BY location_desc
    ORDER BY total_crimes DESC
    LIMIT 10
""").df()

location_concentration
```

Out[29]:

	location_desc	total_crimes	pct_of_total
0	STREET	29321	29.48
1	APARTMENT	18480	18.58
2	RESIDENCE	11807	11.87
3	SIDEWALK	5254	5.28
4	PARKING LOT / GARAGE (NON RESIDENTIAL)	3581	3.60
5	SMALL RETAIL STORE	2927	2.94
6	ALLEY	2324	2.34
7	RESTAURANT	2197	2.21
8	DEPARTMENT STORE	1737	1.75
9	VEHICLE NON-COMMERCIAL	1508	1.52

I also analyzed crime concentration.

Top police districts include

- Chicago Lawn
- Near West
- Gresham.

For location types, the most common locations are:

- Street
- Apartments
- Residences.

Streets alone account for about 29% of crimes.

In [30]:

```
# Week 7: EXPLAIN and filter-early principle
print('=== Plan WITH early filter ===')
plan = con.execute("""
    EXPLAIN
    SELECT district, COUNT(*) AS n
    FROM fact_crime_events
    WHERE district IS NOT NULL
    GROUP BY district
    ORDER BY n DESC
""").df()
print(plan['explain_value'].iloc[0][:1500])
```

```
=== Plan WITH early filter ===
┌──────────────────────────────┐
│           PROJECTION         │
│    ──────────────────────    │
│__internal_decompress_integ│
│      ral_integer(#0, 1)      │
│              #1              │
│                              │
│           ~0 rows            │
└──────────────────────────────┘
               │
┌──────────────────────────────┐
│           ORDER_BY           │
│    ──────────────────────    │
│      count_star() DESC       │
└──────────────────────────────┘
               │
┌──────────────────────────────┐
│           PROJECTION         │
│    ──────────────────────    │
│__internal_compress_integra│
│      l_utinyint(#0, 1)       │
│              #1              │
│                              │
│           ~24 rows           │
└──────────────────────────────┘
               │
┌──────────────────────────────┐
│           PROJECTION         │
│    ──────────────────────    │
│__internal_decompress_integ│
│      ral_integer(#0, 1)      │
│              #1              │
│                              │
│           ~24 rows           │
└──────────────────────────────┘
               │
┌──────────────────────────────┐
│    PERFECT_HASH_GROUP_BY     │
│    ──────────────────────    │
│          Groups: #0          │
│                              │
│         Aggregates:          │
│         count_star()         │
└──────────────────────────────┘
               │
┌──────────────────────────────┐
│           PROJECTION         │
│    ──────────────────────    │
│           district           │
│                              │
│         ~20,000 rows         │
└──────────────────────────────┘
               │
┌──────────────────────────────┐
│           PROJECTION         │
│    ──────────────────────    │
```

In [31]:
```python
# Week 6: CSV vs Parquet benchmark
import time
```

```
t0 = time.perf_counter()
con.execute('SELECT primary_type, COUNT(*) FROM raw_crimes GROUP BY primary_
csv_time = time.perf_counter() - t0

t0 = time.perf_counter()
con.execute("SELECT crime_type, COUNT(*) FROM read_parquet('data/curated/fac
parquet_time = time.perf_counter() - t0

pd.DataFrame({
    'Format': ['CSV (raw_crimes)', 'Parquet (fact_crime_events)'],
    'Query Time (s)': [round(csv_time, 4), round(parquet_time, 4)]
})
```

Out[31]:

	Format	Query Time (s)
0	CSV (raw_crimes)	0.0035
1	Parquet (fact_crime_events)	0.0099

This section demonstrates performance concepts from the course.

I used EXPLAIN to inspect query plans and apply the filter-early principle to optimize queries.

I also compared CSV vs Parquet formats. Parquet is generally faster for analytical queries.

Section 5 - Analysis and Visualization

The visualizations summarize the analysis:

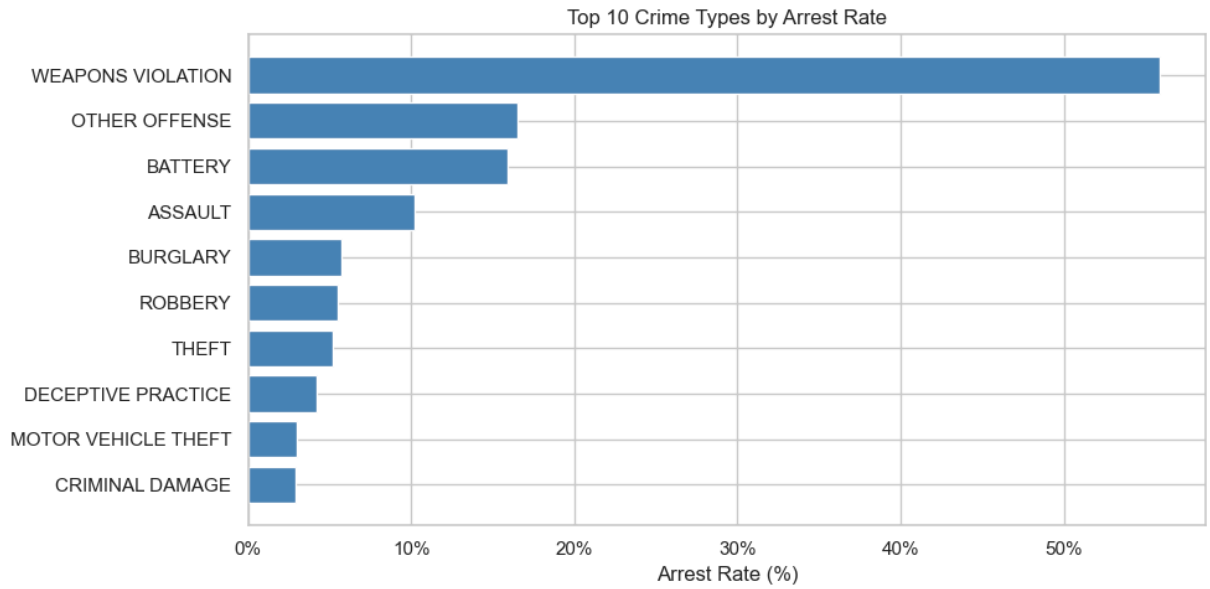
- Crime types by arrest rate
- Average crime by day of week, which increases toward weekends.
- Monthly crime totals
- Scatter plot of crime vs 311 backlog, confirming weak correlation.
- Top police districts by crime volume
- Top location types for crimes.

These charts help clearly illustrate the patterns found in the data.

In [32]:

```
# Figure 1: top crime types by arrest rate
fig, ax = plt.subplots(figsize=(10, 5))
plot_df = crime_type_rates.sort_values('arrest_rate_pct', ascending=True)
ax.barh(plot_df['crime_type'], plot_df['arrest_rate_pct'], color='steelblue'
ax.set_title('Top 10 Crime Types by Arrest Rate')
ax.set_xlabel('Arrest Rate (%)')
ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{x:.0f}%')
plt.tight_layout()
plt.show()
```

```
Task was destroyed but it is pending!
task: <Task pending name='Task-204' coro=<_async_in_context.<locals>.run_in_
context() done, defined at /Library/Frameworks/Python.framework/Versions/3.1
3/lib/python3.13/site-packages/ipykernel/utils.py:57> wait_for=<Task pending
name='Task-205' coro=<Kernel.shell_main() running at /Library/Frameworks/Pyt
hon.framework/Versions/3.13/lib/python3.13/site-packages/ipykernel/kernelbas
e.py:597> cb=[Task.task_wakeup()]> cb=[ZMQStream._run_callback.<locals>._log
_error() at /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.1
3/site-packages/zmq/eventloop/zmqstream.py:563]>
Task was destroyed but it is pending!
task: <Task pending name='Task-205' coro=<Kernel.shell_main() running at /Li
brary/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-package
s/ipykernel/kernelbase.py:597> cb=[Task.task_wakeup()]>
```

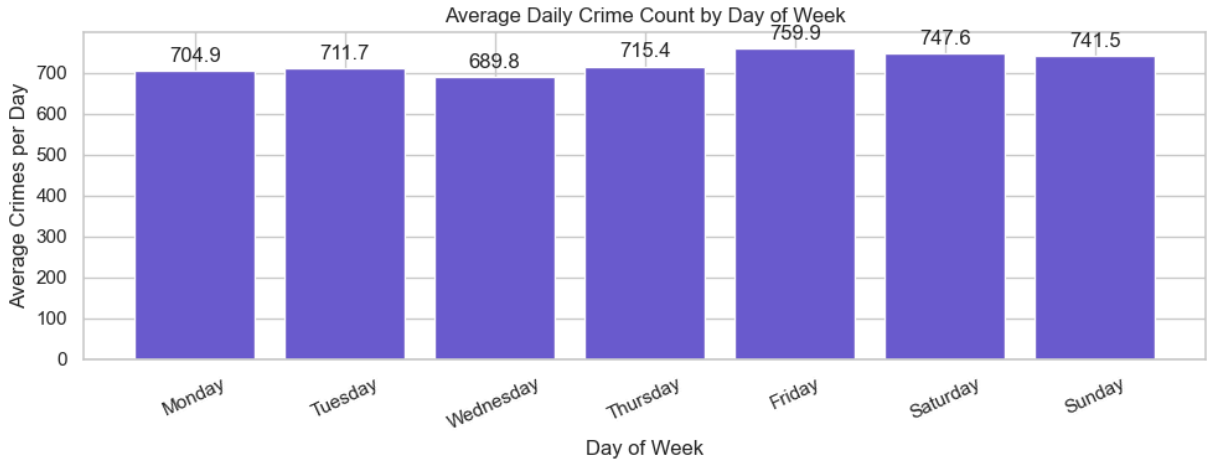


Crime types by arrest rate.

Shows that weapons violations have the highest arrest probability.

In [33]:
```
# Figure 2: daily crime counts by day of week
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturc
dow_df = daily_crime.groupby('day_of_week', as_index=False)['crime_count'].m
dow_df['day_of_week'] = pd.Categorical(dow_df['day_of_week'], categories=day
dow_df = dow_df.sort_values('day_of_week')

fig, ax = plt.subplots(figsize=(10, 4))
bars = ax.bar(dow_df['day_of_week'], dow_df['crime_count'], color='slateblue
ax.bar_label(bars, fmt='%.1f', padding=3)
ax.set_title('Average Daily Crime Count by Day of Week')
ax.set_xlabel('Day of Week')
ax.set_ylabel('Average Crimes per Day')
ax.tick_params(axis='x', rotation=25)
plt.tight_layout()
plt.show()
```

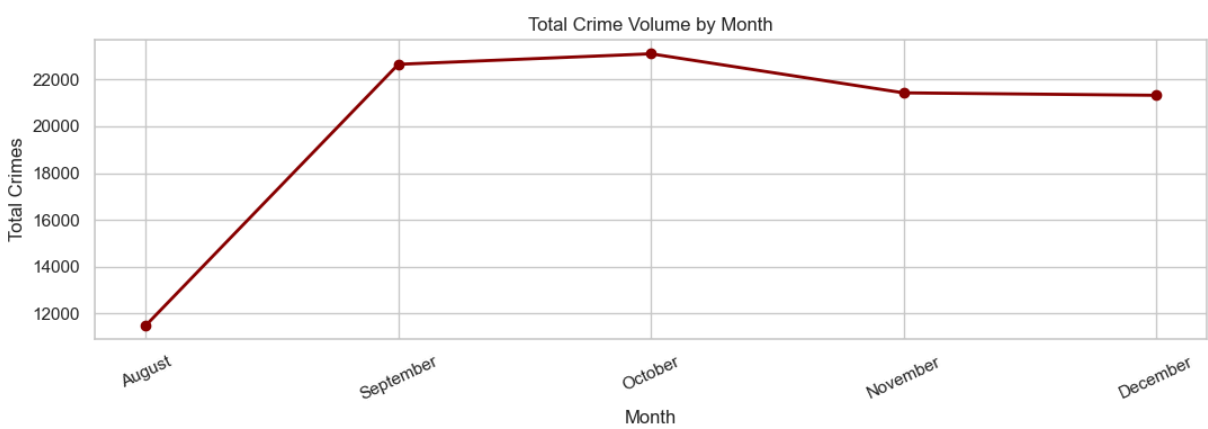


Average crime counts by day of week.

Crime increases toward weekends, especially Friday and Saturday.

In [34]:
```
# Figure 3: monthly crime totals
month_order = ['January', 'February', 'March', 'April', 'May', 'June', 'July
month_df = daily_crime.groupby('month_name', as_index=False)['crime_count'].
month_df['month_name'] = pd.Categorical(month_df['month_name'], categories=m
month_df = month_df.sort_values('month_name')

fig, ax = plt.subplots(figsize=(11, 4))
ax.plot(month_df['month_name'], month_df['crime_count'], marker='o', color='
ax.set_title('Total Crime Volume by Month')
ax.set_xlabel('Month')
ax.set_ylabel('Total Crimes')
ax.tick_params(axis='x', rotation=25)
plt.tight_layout()
plt.show()
```

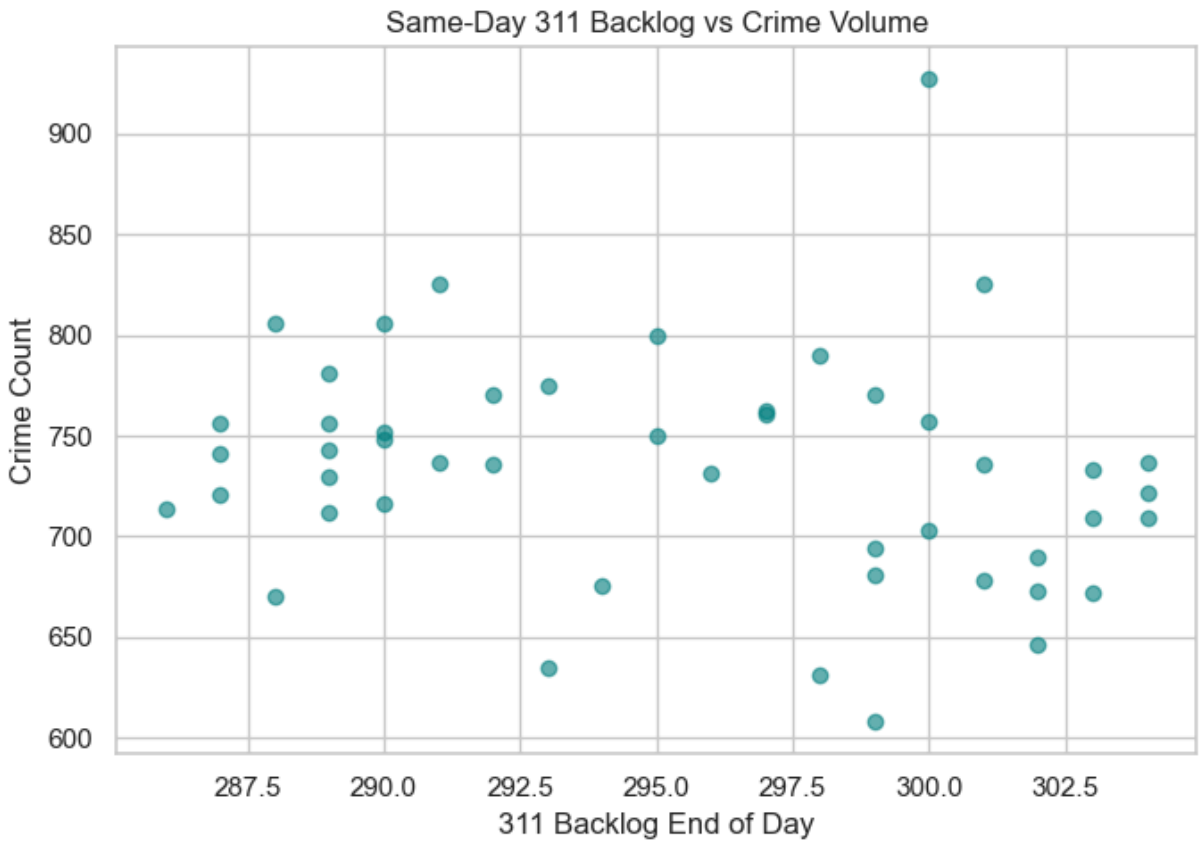


Crime totals by month.

The chart shows seasonal fluctuations in crime patterns.

In [35]:
```
# Figure 4: same-day crime count vs 311 backlog
fig, ax = plt.subplots(figsize=(7, 5))
ax.scatter(crime_311_daily['backlog_end_of_day'], crime_311_daily['crime_cou
ax.set_title('Same-Day 311 Backlog vs Crime Volume')
ax.set_xlabel('311 Backlog End of Day')
ax.set_ylabel('Crime Count')
plt.tight_layout()
plt.show()

corr_df
```



Out[35]:

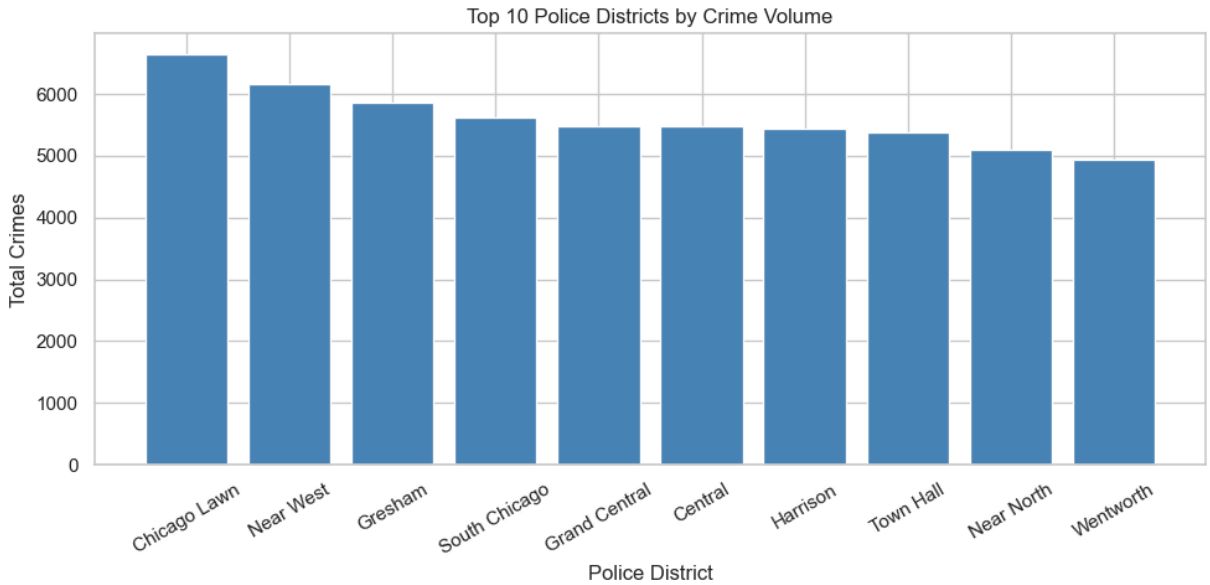
	crime_backlog_corr
0	-0.19532

Scatter plot comparing crime counts and 311 backlog.

The scattered pattern confirms the weak correlation found earlier.

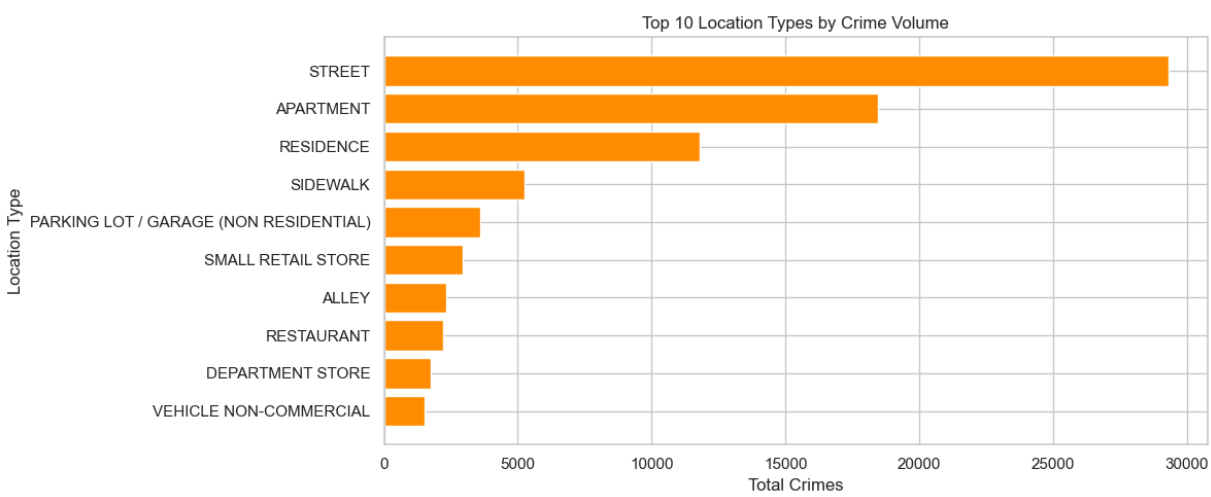
In [36]:
```
# Figure 5: district crime concentration
fig, ax = plt.subplots(figsize=(10, 5))
bars = ax.bar(district_concentration['district_name'], district_concentratic
ax.set_title('Top 10 Police Districts by Crime Volume')
ax.set_xlabel('Police District')
ax.set_ylabel('Total Crimes')
ax.tick_params(axis='x', rotation=30)
```

```
plt.tight_layout()
plt.show()
```



Districts like Chicago Lawn and Near West appear most frequently.

In [37]:
```
# Figure 6: location-type concentration
fig, ax = plt.subplots(figsize=(12, 5))
plot_loc = location_concentration.sort_values('total_crimes', ascending=True
ax.barh(plot_loc['location_desc'], plot_loc['total_crimes'], color='darkorar
ax.set_title('Top 10 Location Types by Crime Volume')
ax.set_xlabel('Total Crimes')
ax.set_ylabel('Location Type')
plt.tight_layout()
plt.show()
```



Top location types for crimes.

Streets and residential areas dominate the dataset.

Section 6 - Reproducibility Log

In [38]:
```
con.execute('SELECT * FROM run_manifest ORDER BY run_id DESC').df()
```

Out[38]:

	run_id	run_ts	raw_crimes_rows	stg_crimes_rows	raw_311_rows	stg_3
0	9	2026-03-10 22:05:59.250793	100000	100000	50000	
1	8	2026-03-10 22:05:22.586273	100000	100000	50000	
2	7	2026-03-10 22:04:21.589931	100000	100000	50000	
3	6	2026-03-10 22:03:52.104231	100000	100000	50000	
4	5	2026-03-10 20:57:29.544717	100000	100000	50000	
5	4	2026-03-10 20:56:58.649668	100000	100000	50000	
6	3	2026-03-10 20:42:22.220677	100000	100000	50000	
7	2	2026-03-10 20:42:00.229797	100000	100000	50000	
8	1	2026-03-10 19:26:46.274057	100000	100000	<NA>	

The pipeline also records runs in a run manifest table, which tracks timestamps and row counts.

This ensures the analysis is reproducible and auditable.

Section 7 - Summary and Limitations

This project demonstrates:

- building a reproducible ETL pipeline
- applying SQL aggregation and analytics
- designing normalized database tables
- analyzing real-world city data.

This analysis shows how database design and SQL analytics can transform raw city datasets into meaningful insights about crime patterns.

In [39]:
```
con.close()
print('Done.')
```

```
Done.
```

In [ ]:

In [ ]: